GraphWeave — Implementation Document

GraphRAG for Multi-Hop Questions

Version 1.0 | 27 October 2026 | Engineering specification for an independently buildable portfolio project

Document control: Treat this document as the project-specific source of truth. If implementation choices change, update the relevant decision record and tests.

Implementation sequence and engineering details

Implement in small, verifiable increments. Do not generate the entire codebase in one unreviewed change. Each phase must end with tests, a build/run check, updated documentation, and a concise report.

1. Ingestion and provenance

Parse documents, preserve document/version/page/section provenance, chunk content, and maintain deterministic IDs so graph facts can be traced back to source evidence.

Implementation checklist:

- Define typed request/response models and service interfaces before connecting UI components.
- Add migrations/indexes and explicit error handling for the data path.
- Add unit tests for normal, boundary, and failure cases.
- Add structured logging and a correlation ID without logging secrets or raw sensitive content.
- Document configuration, trade-offs, and operational behavior.

2. Entity and relation extraction

Use schema-constrained LLM extraction with validated entity types, relation types, confidence, source chunk IDs, and extraction model/version. Treat extracted facts as candidates, not unquestionable truth.

Implementation checklist:

- Define typed request/response models and service interfaces before connecting UI components.
- Add migrations/indexes and explicit error handling for the data path.
- Add unit tests for normal, boundary, and failure cases.
- Add structured logging and a correlation ID without logging secrets or raw sensitive content.
- Document configuration, trade-offs, and operational behavior.

3. Entity resolution

Normalize names and aliases, merge only with explicit confidence rules, preserve aliases, and support review of ambiguous matches. Avoid merging people or organizations solely because names are similar.

Implementation checklist:

- Define typed request/response models and service interfaces before connecting UI components.
- Add migrations/indexes and explicit error handling for the data path.
- Add unit tests for normal, boundary, and failure cases.
- Add structured logging and a correlation ID without logging secrets or raw sensitive content.
- Document configuration, trade-offs, and operational behavior.

4. Knowledge graph

Store entities and typed edges in Neo4j. Include tenant or corpus scope, source chunk references, timestamps, confidence, and extraction provenance on relationships.

Implementation checklist:

- Define typed request/response models and service interfaces before connecting UI components.
- Add migrations/indexes and explicit error handling for the data path.
- Add unit tests for normal, boundary, and failure cases.
- Add structured logging and a correlation ID without logging secrets or raw sensitive content.
- Document configuration, trade-offs, and operational behavior.

5. Hybrid retrieval planner

Retrieve seed chunks/entities by vector similarity, expand graph neighborhoods with bounded depth/fan-out, optionally use relation/path queries, and fuse graph and vector evidence with deduplication.

Implementation checklist:

- Define typed request/response models and service interfaces before connecting UI components.
- Add migrations/indexes and explicit error handling for the data path.
- Add unit tests for normal, boundary, and failure cases.
- Add structured logging and a correlation ID without logging secrets or raw sensitive content.
- Document configuration, trade-offs, and operational behavior.

6. Answer synthesis

Build a compact evidence bundle containing source snippets and relationship paths. Generate answers with citations and distinguish directly stated facts from inferred connections.

Implementation checklist:

- Define typed request/response models and service interfaces before connecting UI components.
- Add migrations/indexes and explicit error handling for the data path.
- Add unit tests for normal, boundary, and failure cases.
- Add structured logging and a correlation ID without logging secrets or raw sensitive content.
- Document configuration, trade-offs, and operational behavior.

7. Graph inspection UI

Allow users to inspect entities, relationships, source provenance, and the paths used for an answer. Scope all graph queries to the authorized corpus/tenant.

Implementation checklist:

- Define typed request/response models and service interfaces before connecting UI components.
- Add migrations/indexes and explicit error handling for the data path.
- Add unit tests for normal, boundary, and failure cases.
- Add structured logging and a correlation ID without logging secrets or raw sensitive content.
- Document configuration, trade-offs, and operational behavior.

8. Evaluation harness

Compare vector-only, graph-only, and hybrid retrieval on a curated multi-hop dataset. Measure answer correctness, evidence recall, citation precision, latency, and token cost.

Implementation checklist:

- Define typed request/response models and service interfaces before connecting UI components.
- Add migrations/indexes and explicit error handling for the data path.
- Add unit tests for normal, boundary, and failure cases.
- Add structured logging and a correlation ID without logging secrets or raw sensitive content.
- Document configuration, trade-offs, and operational behavior.

Detailed request flow

1. Ingest document and preserve stable document, version, chunk, and provenance identifiers.
2. Extract entities and typed relationships using a schema-constrained adapter; validate types and source references.
3. Resolve canonical entities conservatively and write nodes/edges idempotently with provenance and scope.
4. Embed chunks and retrieve vector seed evidence for the question.
5. Expand bounded graph neighborhoods and paths from seed entities; filter all results by scope.
6. Fuse and deduplicate graph and vector evidence, rank it, and assemble a compact evidence bundle.
7. Generate answer with citations to source chunks and explain uncertainty when the evidence path is incomplete.

Configuration and environment

- Provide .env.example with DATABASE_URL, REDIS_URL when used, model provider selection, model names, embedding model, API keys as blank placeholders, log level, and feature flags.
- Validate required settings at startup and fail with actionable errors that do not print secret values.
- Keep local defaults suitable for development only; document production secret management separately.
- Provide deterministic fake model and embedding adapters for CI tests so the core suite does not require paid API calls.

Test strategy

- Unit tests: normalization, policy decisions, metadata validation, ranking/fusion, error mapping, cost calculations, and schema validation.
- Integration tests: PostgreSQL migrations, vector retrieval, Redis/Neo4j interactions where applicable, and API workflows.
- Security tests: authorization boundaries, prompt injection handling, unsafe cache reuse, malformed payloads, and sensitive logging.
- Evaluation tests: fixed dataset, baseline comparison, documented metrics, and regression thresholds.
- Frontend tests: key dashboard states, validation, permission errors, empty states, and API error handling.
- Smoke test: clean Docker Compose startup, health checks, sample ingestion, one query, and dashboard data.

Deployment and operations

- Use Docker Compose for local development with health checks and persistent volumes where required.
- Include migration and seed commands; avoid implicit schema changes on every request.
- Set request timeouts and bounded retries; make ingestion jobs idempotent.
- Document backup/restore expectations, data deletion, cache invalidation, logs/metrics, and known scaling limits.
- Provide a demo script and troubleshooting section.